

软件工程 综合实验报告

实验名称 血潮守望

姓 名 _____

学 号 _____

班 级 计算机 2301 班

教 师 李恒老师

日 期 2026 年 9 月

软件工程综合实验报告

一、实验目的和要求

本实验以 Godot 游戏项目《血潮守望 Crimson Tide》为对象，综合运用软件工程中的需求分析、模块划分、接口设计、实现、测试与交付方法，形成可运行、可验证的多人合作游戏系统。报告重点说明真实源码中的设计取舍及其验证结果。

实验要求包括：明确玩家从进入游戏到结算的业务流程；把地图、战斗、物品、联机和存档拆分为职责清楚的模块；给出关键数据结构与异常处理方法；通过自动化测试和运行截图验证主要功能；说明当前版本的限制与后续改进方向。报告按模板的九个栏目展开，关键代码只作原理说明，不整段复制。

二、实验内容

项目实现一款以“搜集、战斗、撤离、成长”为核心循环的桌面游戏。玩家可单人游玩，也可在局域网使用 2 至 4 人 ENet 房间；仓库还包含账号、云存档及服务器房间方案。进入营地后选择角色、装备与天赋，远征时探索六区地图、清理据点、搜索物资、应对血潮收缩和阶段首领，最终撤离并结算资源。

工程工作覆盖：一是依据固定种子生成地图与可通行路径；二是实现人物移动、武器攻击、敌人与首领行为；三是建立背包格子、装备、搜刮与局内外物资规则；四是实现房主权威模拟、状态同步与结算；五是保存个人成长和可选云端进度；六是将二维玩法坐标映射到 3D 场景，同时继续展示 2D 角色精灵。

三、开发环境

表 1 开发与运行环境

项目	配置或说明
引擎与语言	Godot 4.7.2 stable; 客户端和房间逻辑使用 GDScript
操作系统与渲染	Windows; Godot OpenGL Compatibility; 项目主视口 1440×900
后端	Python 标准库 HTTP 服务、SQLite; 房间工作进程调用 Godot
网络	ENet UDP 实时同步; 账号与云存档通过 HTTP API, 公网应由 HTTPS 代理保护

项目	配置或说明
工程入口	project.godot → scenes/boot.tscn；可用项目内 Godot 可执行文件运行
测试方式	Godot headless 脚本测试；图形截图与多进程联机测试记录保存在 build 目录

当前源码包含新的 3D 地图显示层。根目录 README 与 MAP-2-5D.md 均说明 dist/CrimsonTide.exe 尚未针对该改造重新导出，因此本报告的“当前版本”指项目源码，而非旧的发行包。

四、系统需求分析

4.1 使用对象和业务流程

主要使用对象为单人玩家、房主、加入房间的队友，以及可选的服务器管理员。玩家目标是在有限时间内搜集战利品、完成据点和首领挑战并安全撤离。局内失败会损失角色背包与身上装备，固定 4×4 次元口袋中的物品保留，形成风险与收益之间的选择。

典型流程为：标题页选择单人或房间 → 营地选人、配置与准备 → 地图探索、搜刮、战斗 → 每日血潮与首领 → 第二天决定撤离或继续挑战 → 结算奖励和经验 → 写入存档并回到营地。网络模式下，房主或专用房间作为权威端处理状态，客户端提交操作并接收快照。

4.2 功能需求

表 2 主要功能需求与验收依据

编号	需求	可观察结果
F1	角色与营地配置	能选角色、装备、购买补给与升级天赋，进入战局后配置生效
F2	地图与探索	相同种子产生一致的六区地图；道路、桥、据点与撤离点可达
F3	战斗与远征	攻击、闪避、技能、首领阶段与血潮规则按时间和状态触发
F4	物品与背包	物品按真实格数占位；能搜索、旋转、搬运、装备和消耗
F5	合作联机	2 至 4 人加入房间；权威端同步移动、战斗、掉落与各人撤离
F6	结算与存档	按撤离或失败结果结算；成长、口袋与仓库下次启动可恢复

编号	需求	可观察结果
F7	在线服务	可注册登录、选择云端同步并以房间号加入服务器房间

4.3 非功能需求和约束

正确性方面，随机地图必须保证关键点可达，背包操作不得造成物品复制、丢失或重叠；网络模式下由权威端决定伤害、掉落和结算。可靠性方面，存档先写临时文件再替换正式文件，并对格式错误或越界数据进行清理。可维护性方面，规则数据、地图、显示、在线服务与测试脚本分别组织。

边界条件包括：局域网 IP 直连没有自动匹配、NAT 打洞和局中重连；公网服务需要自行部署并配置真实域名，仓库中的 example.com 是占位地址；实时 ENet 流量尚未加密。当前 2.5D 展示只改变显示层，人物仍在单一地面层活动，没有跳跃或多层导航。

五、系统概要设计

5.1 总体架构

系统采用“输入与界面—权威游戏会话—规则与数据—显示和持久化”的分层结构。scripts/main.gd 负责页面、交互和输入；scripts/session.gd 保存战局状态并执行权威模拟；scripts/catalog.gd 提供角色、武器、装备和容器规则；scripts/ruins.gd 与 scripts/expedition.gd 分别处理地图和远征进程；scripts/battlefield.gd、scripts/world_3d.gd 负责可视化；scripts/profile.gd 与 scripts/online_service.gd 处理本地及可选在线进度。

表 3 核心模块与职责

模块	主要职责	关键接口或文件
界面层	菜单、营地、背包、战术地图、HUD 与操作反馈	main.gd、battlefield.gd
会话层	单人/房间、玩家状态、命令校验、战斗与结算	session.gd
规则层	地图生成、远征日程、物品和数值规则	ruins.gd 、 expedition.gd 、 catalog.gd
显示层	正交相机、3D 场景和 2D 角色精灵投影	world_3d.gd、battlefield.gd

模块	主要职责	关键接口或文件
持久化层	本地 JSON、账号和云存档版本管理	profile.gd、online_service.gd、 server/app.py

5.2 数据流与状态边界

输入事件由界面转换为移动、攻击、交互等命令。单人模式直接交给本地 TideSession；联机模式传给房主或专用房间。会话层依据地图碰撞、角色属性和物品规则更新状态，再把快照与事件供渲染层显示。个人成长由 Profile 写入 user://profile.json；账号模式可将经校验的 JSON 存档上传至 SQLite，使用 revision 识别版本冲突。

地图逻辑与视觉坐标分离：Ruins 使用 6400×4800 的二维世界坐标负责碰撞、视线和路径；World3D 把二维 (x, y) 映射为三维 (x, 0, z)，通过正交相机渲染。人物和敌人以 Sprite3D 显示，战斗判定仍按原二维规则执行，从而降低显示改造对玩法、网络协议与测试的影响。

六、系统详细设计

6.1 地图生成与可达性

Ruins.generate(seed) 先以固定种子初始化随机数，再构建六个区域、多边形海岸线、河流、三座桥、道路和 18 个据点。关键据点按设计坐标放置，随机性主要用于补给和外围装饰，使任务结构稳定且每局细节不同。blocked、move 与 clear_line 分别提供碰撞、移动约束和视线判断；30 种种子的路径检查用于降低“出生后无法到达目标”的风险。

6.2 战斗与远征状态机

TideSession 在物理帧更新玩家、怪物、投射物和互动，只有 authority() 为真时执行决定性规则。Expedition 维护天数、计时器、首领生成和胜利选择：探索日为 300 秒，约第 180 秒开始血潮收缩，第 300 秒进入当日首领阶段；第一天不能撤离，第二天胜利后可选择撤离或继续，第三天进入终局挑战。技能预警锁定目标时的位置和方向，再按延迟与范围结算命中，保证玩家可观察、可躲避。

战斗系统把临时武器与局内战利品武器分开。角色出生时有专属低强度临时武器；局内武器必须先拾取并装备。卸下武器或阵亡时恢复临时武器，避免角色进入“无武器”非法状态。数值表集中在 Catalog，战斗、动画和音效通过武器族读取同一分类，减少重复分支。

6.3 网格背包和搜刮事务

次元口袋固定为 4×4 ；局内背包按白、绿、蓝、紫、金、红六档扩展，容器条目标录种类、位置、旋转、数量和品质等字段。Catalog.can_place 检查边界和重叠，clean_container 在读档时过滤不合法数据，place_item 与相关搬运逻辑在空位不足时拒绝操作。武器占 2×2 ，部分消耗品为 1×2 ，遗物可占 2×2 或更大，因此旋转与自动找位均须按真实尺寸处理。

搜索容器不是一次性打开全部内容：交互开始后物品逐件显露，玩家可拖入背包或口袋；离开范围会暂停搜索。装备和消耗品的使用由会话层更新状态，界面只提供入口。阵亡时丢弃局内背包与身上装备，次元口袋保留；成功撤离时按携带物品计算个人收益，并与团队奖励合并结算。

6.4 联机、账号和存档

IP 直连使用 ENet 房主权威模型。玩家加入后通过 RPC 发送操作，权威端广播状态与事件；每名玩家可独立撤离，全部玩家结束后统一结算。服务器房间通过 Python API 创建房间并启动独立的 Godot 进程，入场凭证用于限定加入者，六位房间号用于邀请。账号密码使用加盐 bcrypt 哈希，服务器以 SQLite 保存账号、令牌哈希和带 revision 的云存档。

本地 Profile 对旧存档进行类型、范围和容器位置校验。保存时先写 profile.json.tmp，再重命名为 profile.json，避免写入中途直接破坏正式文件。在线同步发生冲突时暂停自动覆盖，并要求玩家选择保留本地还是云端；下载前备份现有本地文件。

6.5 2.5D 显示实现

World3D 管理正交相机、灯光、地面、墙体、桥梁、建筑和宝箱，地图对象或种子变化时重建，其余帧复用网格与材质。Battlefield 将人物、普通敌人和首领放入可复用的 Sprite3D 池，用脚底锚点与深度测试实现和建筑的前后遮挡；鼠标射线与地面求交得到瞄准点，血条和技能预警再投影到二维覆盖层。人物仅绕竖轴面向相机，以免身体向后倾入岩台。

七、运行结果及分析

7.1 当前源码的直接验证

在 Windows 环境使用项目内 Godot 4.7.2 stable，以 `--headless --path . --script` 运行三个测试脚本。2026 年 9 月 26 日的实际输出如下。测试进程均以退出码 0 结束，但 headless 结果主要证明规则和场景结构正确，不能替代完整的多人公网游玩与人工画面评审。

表 4 本次直接运行的测试结果

测试脚本	检查数	失败数	验证重点
tests/systems.gd	10812	0	容器、存档、地图、结算等系统规则
tests/combat.gd	78	0	临时武器、装备后换用和战斗判定
tests/world_3d.gd	32	0	投影、场景重建、遮挡与对象复用

项目既有 TEST-REPORT.md 还记录了 2026 年 9 月 23 日的 6176 项规则检查及本机双进程联机通过；这些数字属于当时版本的验证历史，不与本次 10812 项相加，也不代表当前源码已完成跨公网压力测试。

7.2 画面和交互结果

项目留存截图展示了格子背包与次元口袋同时显示、装备槽和物品信息，以及 3D 场景中角色靠近岩台时的遮挡关系。图 1 为背包交互截图；图 2 为当前显示层的深度修复截图。图像是项目测试留存，用于说明界面结构和显示效果。



图 1 背包、次元口袋与装备栏的界面测试截图



图 2 角色精灵在 3D 岩台前的深度显示测试截图

从图 1 可见，角色背包和永久口袋分别显示，右侧装备栏提供明确的穿戴状态；从图 2 可见，人物脚底与地面关系稳定，身体没有被前景岩台错误截断。截图验证的是特定运行场景，画面品质仍需结合完整战局人工检查。

7.3 结果边界

当前源码的 2.5D 地图与原有玩法坐标共存，规则测试已通过，但 dist 中的旧 EXE 未重新导出。在线服务的示例域名尚未部署；若要完成公网联机演示，还需准备服务器、HTTPS 证书、UDP 端口及实际网络测试。

八、遇到的问题及解决情况

表 5 关键问题、定位与处理

问题	原因分析	处理与验证
背包拖拽图标不跟随鼠标	图标只在背包内容变化时重建，未逐帧更新位置	改为常驻覆盖节点，每帧同步拖拽图标及落点；项目 UI 回归截图验证
物品读档或搬运后字段丢失	按 kind 重建条目时漏掉数量、品质和武器索引	容器清理与搬运保留完整字段；系统回归覆盖存档往返和换包
搜刮计时不能推进	交互通道逐帧清空搜索计时器	分离搜索计时状态；验证逐件显露和暂停恢复
大快照超过 UDP MTU	不可靠消息直接发送未压缩状态	使用独立通道发送压缩可靠快照；本机联机回归验证
角色贴近岩台时身体被截断	全朝向 billboard 使上半身向后倾入几何体	精灵只绕 Y 轴面向相机并校准脚底；深度图形测试复现及修复

仍需跟踪的问题包括：2.5D 显示层暂不支持多层高度玩法；部分旧图形测试夹具与当前规则不一致；少数第四角色音频源文件缺失会影响完整主场景加载。报告没有把这些项目写成已解决事项，后续应先补齐资源和测试夹具，再执行全流程图形与多人回归。

九、总结

本实验完成了从需求、架构、详细设计到验证的完整软件工程分析。项目把多人权威会话、地图与远征规则、网格背包、持久化和 2.5D 显示拆分为相对独立的模块，使游戏循环既能运行，也能通过脚本和截图逐项检查。当前直接运行的系统、战斗与 3D 显示测试全部通过，说明核心规则和显示映射具备可验证性。

后续应补齐图形与公网联机回归、修复缺失资源，并重新导出与源码一致的发行包。